

Monte Carlo for delta hedging of AAO. Code explanation

Zakhar Nefedov

Contents

1	Notation and variables	2
2	Notes	2
3	Inputs	3
4	Hedging block	3
4.1	Conditional Brownian motion	3
4.2	Delta hedge	4
5	Geometric asian options	5
5.1	Closed form	5
5.1.1	Price	5
5.1.2	Delta	6
5.2	Path-dependent	7
5.2.1	Price	7
5.2.2	Delta	8
6	Arithmetic asian options	9
6.1	Price	9
6.2	Delta	10
7	Variance reduction	10
7.1	Antithetic variates	10
7.2	Kemna-Vorst method	10
A	Appendix	12
A.1	Proof of 5.1.1	12
A.2	Linear approximation of Brownian motion (5.2.1)	13
A.3	Proof of 5.2.2	13

1 Notation and variables

- i $\widetilde{\mathbb{E}}, \widetilde{\mathbb{P}}$ - expectation, probability under standard risk-neutral probability measure
- ii $\widetilde{W}(t)$ - stochastic process that is a Brownian motion under $\widetilde{\mathbb{P}}$. I.e $d\widetilde{W}(t)d\widetilde{W}(t) = t$, $\widetilde{\mathbb{E}}\widetilde{W}(t) = 0$ and $\widetilde{\text{Var}}(\widetilde{W}(t)) = t$
- iii $\mathbb{I}_{\{A\}}$ - indicator function for condition A . I.e $\mathbb{I}_{\{A\}} = 1$ if A is true and 0 otherwise.
- iv $A^{(G)}(t)$ - price of the geometric asian option at time t
- v $\Delta^{(G)}(t)$ - delta of the geometric asian option at time t
- vi $A^{(A)}(t)$ - price of the arithmetic asian option at time t
- vii $\Delta^{(A)}(t)$ - delta of the arithmetic asian option at time t
- viii T - time of expiration ($1 = 1$ year)
- ix R - risk-free interest rate
- x σ - implied volatility of stock
- xi K - strike price
- xii $S(t)$ - stock price at time t

2 Notes

- i Implied volatility σ and interest rate R are assumed to be constant. Dividend payment is assumed to be zero.
- ii Brownian motion is modeled using n discrete increments, equal to random sample based of normal random variable with mean 0, s.d. $\frac{T}{n}$. (antithetic variates are used, see 7.1)
- iii At time t , price and delta are equal to expectation conditional on filtration $\mathcal{F}(t)$, i.e. $A^{(A)}(t) = \dots \widetilde{\mathbb{E}}[\dots | \mathcal{F}(t)]$. But, all of the formulas given later are for $t = 0$. This is because underlying Brownian motion is changed to include filtration, and hence, formulas remain the same for every time increment
- iv Stock price as function of time is taken as standard geometric Brownian motion $S(t) = S(0) \exp \left\{ \left(R - \frac{1}{2}\sigma \right) t + \sigma \widetilde{W}(t) \right\}$
- v All the calculations in the code are vectorised using `numpy`, but formulas are given as for single simulated path of Brownian motion.

3 Inputs

Simulation takes the following parameters in order:

- i T - as in 1, viii
- ii n - number of discrete steps in the simulated Brownian motion.
- iii M - number of simulation runs per rebalancing step.
- iv $R, \sigma, K, S(0)$ - parameters as in 1, ix - xii
- v $\mathbf{S}_T$ - vector of real stock prices from $t = 0$ to $t = T$. Hence, as is, simulation only works on past data, which was done for testing. But, can easily be adapted to take incoming stock price data

```
def __init__(self, in_final_t, in_n_brownian_stp, in_n_sim_run,
             in_interest_rt, in_volatility, in_strike_prc,
             in_initial_prc, in_m_stock_prc):
    _n_sim_used_halfed = int(0.5 * in_n_sim_run)
    self.l_parameters = [in_final_t, in_n_brownian_stp, in_n_sim_run,
                        in_interest_rt, in_volatility, in_strike_prc, in_initial_prc]
    self.m_stock_prc =
        numpy.concatenate((numpy.array([in_initial_prc]), in_m_stock_prc))

    self.m_brownian = numpy.concatenate((numpy.zeros((_n_sim_used_halfed,1)),
        numpy.cumsum(numpy.multiply(numpy.random.randn(
            _n_sim_used_halfed, in_n_brownian_stp)),
            numpy.sqrt(in_final_t / in_n_brownian_stp))), axis = 1)), axis = 1)
    self.m_brownian = numpy.concatenate((self.m_brownian,
        numpy.negative(self.m_brownian)), axis = 0)
    self.m_timesteps = numpy.array([numpy.concatenate(
        ([0], numpy.cumsum(numpy.full((1, in_n_brownian_stp),
            in_final_t / in_n_brownian_stp))))]).T
```

Here, `self.timesteps` is a column vector of time increments, `self.m_brownian` is a $M \times n$ matrix, with each row representing separate simulated run, each column representing discrete step (see 2, ii and 7.1 for more information)

4 Hedging block

4.1 Conditional Brownian motion

Function `f_stock_cond` converts stock prices known up to time t to their underlying Brownian motion.

$$W(t) = \frac{1}{\sigma} \log \left(\frac{S(t)}{S(0)} \right) - \frac{t}{\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$$

```

def f_stock_cond(self, in_n):
    _m_stock_cond = self.m_stock_prc[:in_n]
    self.m_brownian_cond = numpy.log(_m_stock_cond
    / self.l_parameters[6] / self.l_parameters[4]
    - (self.m_timesteps.T[0][:in_n] / self.l_parameters[4]))
    * (self.l_parameters[3] - 0.5 * self.l_parameters[4] ** 2)

```

Function `f_brownian_upd` replaces first `in_n` steps of each simulated Brownian motion path with real stock price data. This has the identical effect to taking conditional expectation (see 2, iii)

```

def f_brownian_upd(self, in_n):
    self.f_stock_cond(in_n)
    _m_brownian_sep = self.m_brownian[:,
    1:self.l_parameters[1] - in_n + 2]
    _m_tiled_brownian_last = numpy.tile( numpy.array(
    [self.m_brownian_cond[-1]]),
    (self.l_parameters[2], self.l_parameters[1] - in_n + 1))
    self.m_brownian_cps = numpy.concatenate(
    (numpy.tile(self.m_brownian_cond, (self.l_parameters[2], 1)),
    _m_tiled_brownian_last + _m_brownian_sep), axis = 1)

```

4.2 Delta hedge

Main function `f_test_recur` performs the delta hedge, returns the final portfolio price.

$$A_{KV}^{(A)}(t) = \Delta_{KV}^{(A)}(t)S(t) + B(t)$$

where $\exp\{\frac{TR}{n}\}$ is accumulated on the cash balance $B(t)$ per time step:

```

def f_test_recur(self):
    self.f_kemna_vorst_block_prc(self.m_brownian)
    self.f_kemna_vorst_block_dlt(self.m_brownian)
    _init_cap = self.aao_kemna_vorst_prc
    _prev_dlt = self.aao_kemna_vorst_dlt
    _debt_ir = _init_cap - self.l_parameters[6] * _prev_dlt
    for i in range(2, self.l_parameters[1] + 2):
        self.f_brownian_upd(i)
        _debt_ir = _debt_ir * math.exp(self.l_parameters[3] * self.l_parameters[0]
        / self.l_parameters[1])
        _m_brownian = self.m_brownian_cps
        self.f_kemna_vorst_block_dlt(_m_brownian)
        _scaled = (self.l_parameters[1] - i + 1) / (self.l_parameters[1] + 1)
        _debt_ir = _debt_ir - self.m_stock_prc[i-1] *
        (_scaled * self.aao_kemna_vorst_dlt - _prev_dlt)
        _prev_dlt = _scaled * self.aao_kemna_vorst_dlt
    return(_scaled * self.aao_kemna_vorst_dlt * self.m_stock_prc[i-1] + _debt_ir)

```

5 Geometric asian options

5.1 Closed form

5.1.1 Price

Geometric mean over continuous function $f(x)$ on the range $(0, T)$ is:

$$\Gamma_{f(x)} = \exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\}$$

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

Solving for expectation, value of the Geometric Asian Option is:

$$A^G(0) = S(0) \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K e^{-RT} \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right)$$

```
f_cfs_gao_prc = lambda self:
    self.l_parameters[6] * self.f_gamma_t()
    * (1 - statistics.NormalDist(
        (self.l_parameters[4] * math.pow(self.l_parameters[0], 2)) / 3,
        math.sqrt(math.pow(self.l_parameters[0], 3) / 3)).cdf(self.f_lambda_t()))
    - K * math.exp(-R * self.l_parameters[0])
    * (1 - statistics.NormalDist(0,
        math.sqrt(math.pow(self.l_parameters[0], 3) / 3)).cdf(self.f_lambda_t()))
```

where $N(x, \mu, \sigma)$ is a cumulative density function for normal RV from $-\infty$ to x , with mean μ and standard deviation σ . Other functions are:

$$\gamma_T = \exp \left\{ -\frac{1}{2} RT - \frac{1}{12} \sigma^2 T \right\}$$

```
f_gamma_t = lambda self:
    math.exp(-0.5 * self.l_parameters[3] * self.l_parameters[0]
    - (pow(self.l_parameters[4], 2) * self.l_parameters[0]) / 12)
```

$$\lambda_T = \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T^2}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$$

```
f_lambda_t = lambda self:
    (math.log(self.l_parameters[5] / self.l_parameters[6])
    * self.l_parameters[0]) / self.l_parameters[4] - (self.l_parameters[3]
    - 0.5 * math.pow(self.l_parameters[4], 2))
    * math.pow(self.l_parameters[0], 2) / (2 * self.l_parameters[4])
```

5.1.2 Delta

$$\begin{aligned}
\Delta^{(G)}(0) &= \frac{\partial A^{(G)}(0)}{\partial S(0)} \\
&= \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) + \frac{\gamma_T}{\sigma} \sqrt{\frac{3}{2\pi T}} \exp \left\{ -\frac{3}{2T^3} \left(\lambda_T^2 - \frac{2}{3} \lambda_T \sigma T^2 + \frac{\sigma^2 T^4}{9} \right) \right\} \\
&\quad - \frac{K e^{-RT}}{\sigma S(0)} \sqrt{\frac{3}{2\pi T}} \exp \left\{ -\frac{3\lambda_T^2}{2T^3} \right\} \\
&= a + b + c
\end{aligned}$$

where:

$$\begin{aligned}
a &= \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) \\
\phi &= \frac{1}{\sigma} \sqrt{\frac{3}{2\pi T}} \\
b &= \gamma_T \phi \exp \left\{ -\frac{3}{2T^3} \left(\lambda_T^2 - \frac{2}{3} \lambda_T \sigma T^2 + \frac{\sigma^2 T^4}{9} \right) \right\} \\
c &= \frac{K e^{-RT}}{S(0)} \exp \left\{ -\frac{3\lambda_T^2}{2T^3} \right\}
\end{aligned}$$

```

def f_cfs_gao_dlt(self):
    _gamma_t = self.f_gamma_t()
    _lambda_t = self.f_lambda_t()
    _term_a = _gamma_t * (1 - statistics.NormalDist((self.l_parameters[4]
        * math.pow(self.l_parameters[0], 2)) / 3,
        math.sqrt(math.pow(self.l_parameters[0], 3) / 3)).cdf(_lambda_t))
    _term_phi = 1 / (self.l_parameters[4]
        * math.sqrt(2 * math.pi * self.l_parameters[0] ** 3 / 3))
    _term_b = _term_phi * _gamma_t * math.exp((-3 / (2 * self.l_parameters[0] ** 3))
        * ((_lambda_t ** 2) - (2 * _lambda_t * self.l_parameters[0] ** 2
        * self.l_parameters[4] / 3) + (self.l_parameters[4] ** 2
        * self.l_parameters[0] ** 4 / 9)))
    _term_c = _term_phi * self.l_parameters[5]
        * math.exp(-self.l_parameters[0] * self.l_parameters[3])
        / self.l_parameters[6] * math.exp((-3 / (2 * self.l_parameters[0] ** 3))
        * (_lambda_t ** 2))
    self.cfs_gao_dlt = _term_a + _term_b - _term_c

```

where γ_T and λ_T are as in [5.1.1](#).

5.2 Path-dependent

5.2.1 Price

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

Expanding:

$$\begin{aligned} A^G(0) &= e^{-RT} \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{2} \sum_{i=0}^{i_T} \frac{w_{t_{i+1}} + w_{t_i}}{n} \right\} - K \right)^+ \right] \\ &= e^{-RT} \tilde{\mathbb{E}} \left[(S(0) \alpha_T \beta_T - K)^+ \right] = e^{-RT} \tilde{\mathbb{E}} \left[(\bar{S}(T) - K)^+ \right] \end{aligned}$$

Hence, under filtration $\mathcal{F}(T)$, this simplifies to:

$$A^{(G)}(0) = e^{-RT} (\bar{S}(T) - K)^+$$

```
def f_pd_gao_prc(self, _in_m_brownian):
    self.f_pd_gao_s_bar(_in_m_brownian)
    self.m_pd_gao_prc = numpy.exp(-self.l_parameters[3] * self.l_parameters[0])
    * numpy.maximum(self.m_pd_gao_s_bar - self.l_parameters[5], 0)
```

where:

$$\bar{S}(T) = S(0) \alpha_T \beta_T$$

```
def f_pd_gao_s_bar(self, _in_m_brownian):
    self.m_pd_gao_s_bar = self.l_parameters[6]
    * numpy.multiply(self.f_pd_gao_alpha(), self.f_pd_gao_beta(_in_m_brownian))
```

$$\alpha_T = \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\}$$

```
f_pd_gao_alpha = lambda self:
    numpy.exp(0.5 * (self.l_parameters[3] - 0.5 * self.l_parameters[4] ** 2)
    * self.l_parameters[0])
```

$$\beta_T = \exp \left\{ \frac{\sigma}{2T} \sum_{i=0}^{i_T} \left(\frac{T}{n} (w_{t_{i+1}} + w_{t_i}) \right) \right\} = \exp \left\{ \frac{\sigma}{2T} \sum_{i=0}^{i_T} s_t \right\}$$

```
f_pd_gao_beta = lambda self:
    numpy.exp(self.l_parameters[4]
    * self.f_matrix_sstep() / (2 * self.l_parameters[0]))
```

$$s_t = \frac{T}{n} (w_{t_{i+1}} + w_{t_i})$$

```
f_matrix_sstep = lambda self:
    numpy.sum((self.m_brownian[:, 1:] + self.m_brownian[:, :-1]), axis = 1)
    * (self.l_parameters[0] / self.l_parameters[1])
```

5.2.2 Delta

Claim that:

$$\frac{\partial \bar{S}(T)}{\partial S(0)} = \frac{\bar{S}(T)}{S(0)}$$

(proof is provided in the appendix [A.3](#)). Hence,

$$\Delta^{(G)}(0) = \frac{\partial}{\partial S(0)} e^{-RT} \tilde{\mathbb{E}}[(\bar{S}(T) - K)^+] = e^{-RT} \tilde{\mathbb{E}} \left[\frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}} \right]$$

```
def f_pd_gao_dlt(self, _in_m_brownian):
    self.f_pd_gao_s_bar(_in_m_brownian)
    self.m_pd_gao_dlt = numpy.exp(-self.l_parameters[3] * self.l_parameters[0])
        * (self.m_pd_gao_s_bar / self.l_parameters[6])
        * (numpy.sign(self.m_pd_gao_s_bar - self.l_parameters[5]) + 1) / 2
```

Where $\bar{S}(T)$ is as in [5.2.1](#).

6 Arithmetic asian options

6.1 Price

Arithmetic asian option (AAO) is defined as:

$$A^{(A)}(0) = e^{-RT} \mathbb{E} \left[\left(\frac{1}{T} \int_0^T S(t) dt - K \right)^+ \right] = e^{-RT} \mathbb{E} [(\bar{S}(T) - K)^+]$$

```
def f_pd_aao_prc(self, _in_m_brownian):
    self.f_pd_aao_s_bar(_in_m_brownian)
    self.m_pd_aao_prc = numpy.array([numpy.exp(-self.l_parameters[3]
        * self.l_parameters[0]) * numpy.maximum(self.m_pd_aao_s_bar
        - self.l_parameters[5], 0)].T
```

$$\bar{S}(T) = \frac{S(0)}{T} \sum_{i=0}^{i_T} \left(\frac{1}{\alpha_i} e^{\beta_i} (e^{\alpha_i t_{(i+1)}} - e^{\alpha_i t_i}) \right)$$

```
def f_pd_aao_s_bar(self, _in_m_brownian):
    self.f_pd_aao_primary_f(_in_m_brownian)
    _m_pd_aao_step = numpy.reciprocal(self.m_alpha_i)
        * numpy.exp(self.m_beta_i)
        * (numpy.exp(numpy.dot(self.m_alpha_i,
            numpy.diag(self.m_timesteps[1:, :].T[0])))
        - numpy.exp(numpy.dot(self.m_alpha_i,
            numpy.diag(self.m_timesteps[:-1, :].T[0]))))
    self.m_pd_aao_s_bar = (self.l_parameters[6] / self.l_parameters[0])
        * numpy.sum((_m_pd_aao_step), axis = 1)
```

where α_i, β_i are in a method `f_pd_aao_primary_f`:

$$\delta_i = \frac{w(t_{(i+1)}) - w(t_i)}{t_{(i+1)} - t_i}$$

$$\alpha_i = \left(R - \frac{1}{2} \sigma^2 + \delta \cdot \sigma \right)$$

$$\beta_i = -\sigma (\delta \cdot t_i - w(t_i))$$

```
def f_pd_aao_primary_f(self, _in_m_brownian):
    _m_delta = (_in_m_brownian[:, 1:] - _in_m_brownian[:, :-1])
        * self.l_parameters[1] / self.l_parameters[0]
    self.m_alpha_i = self.l_parameters[3]
        - 0.5 * self.l_parameters[4] ** 2
        + _m_delta * self.l_parameters[4]
    self.m_beta_i = -self.l_parameters[4]
        * (numpy.dot(_m_delta, numpy.diag(self.m_timesteps[:-1, :].T[0]))
        - _in_m_brownian[:, :-1])
```

6.2 Delta

Delta of AAO is defined as:

$$\Delta(0) = \frac{\partial}{\partial S(0)} e^{-RT} \tilde{\mathbb{E}} \left[(\bar{S}(T) - K)^+ \right] = e^{-RT} \tilde{\mathbb{E}} \left[\frac{\partial \bar{S}(T)}{\partial S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}} \right] = e^{-RT} \tilde{\mathbb{E}} \left[\frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}} \right]$$

Hence, under filtration $\mathcal{F}(T)$:

$$\Delta(0) = e^{-RT} \frac{\bar{S}(T)}{S(0)} \mathbb{I}_{\{\bar{S}(T) > K\}}$$

```
def f_pd_aao_dlt(self, _in_m_brownian):
    self.f_pd_aao_s_bar(_in_m_brownian)
    self.m_pd_aao_dlt = numpy.exp(-self.l_parameters[3] * self.l_parameters[0])
        * self.m_pd_aao_s_bar / self.l_parameters[6]
        * (numpy.sign(self.m_pd_aao_s_bar - self.l_parameters[5]) + 1) / 2
```

Where $\bar{S}(T)$ is as in 6.1.

7 Variance reduction

7.1 Antithetic variates

For n simulated Brownian motions, $\frac{n}{2}$ are simulated randomly:

$$W_1 = \begin{pmatrix} w_{11} & \cdots & w_{1T} \\ \vdots & \ddots & \vdots \\ w_{\frac{n}{2}1} & \cdots & w_{\frac{n}{2}T} \end{pmatrix}$$

and the other $\frac{n}{2}$ runs are taken as $W_2 = -W_1$. W_1 and W_2 are perfectly negatively correlated, reducing variance for Monte Carlo simulations run.

```
self.m_brownian = numpy.concatenate(
    (numpy.zeros((_n_sim_used_halfed,1)),
    numpy.cumsum(numpy.multiply(numpy.random.randn(
    _n_sim_used_halfed, in_n_brownian_stp),
    numpy.sqrt(in_final_t / in_n_brownian_stp))), axis = 1)), axis = 1)
self.m_brownian = numpy.concatenate(
    (self.m_brownian, numpy.negative(self.m_brownian)), axis = 0)
```

7.2 Kemna-Vorst method

This is similar for both price and delta.

As arithmetic and geometric mean are correlated, can utilise the fact that there is closed-form solution for price and delta of geometric asian options.

$$A_{KV}^{(A)} = A^{(A)} - c(A^{(G)} - \tilde{\mathbb{E}}A^{(G)})$$

where:

$$c = \frac{\text{Cov}(A^{(A)}, A^{(G)})}{\text{Var}(A^{(G)})}$$

```
def f_kemna_vorst_block_prc(self, _in_brownian):
    self.f_pd_gao_prc(_in_brownian)
    self.f_pd_aao_prc(_in_brownian)
    _gao_prc_mean = numpy.mean(self.m_pd_gao_prc)
    _aao_prc_mean = numpy.mean(self.m_pd_aao_prc)
    _c = numpy.cov(self.m_pd_aao_prc.T[0], self.m_pd_gao_prc)[0,1]
        / numpy.var(self.m_pd_gao_prc, ddof = 1)
    self.aao_kemna_vorst_prc = _aao_prc_mean - _c
        * (_gao_prc_mean - self.f_cfs_gao_prc())
```

Identically for delta:

$$\Delta_{KV}^{(A)} = \bar{\Delta}^{(A)} - c(\bar{\Delta}^{(G)} - \tilde{\mathbb{E}}\Delta^{(G)})$$

where:

$$c = \frac{\text{Cov}(\Delta^{(A)}, \Delta^{(G)})}{\text{Var}(\Delta^{(G)})}$$

```
def f_kemna_vorst_block_dlt(self, _in_m_brownian):
    self.f_pd_gao_dlt(_in_m_brownian)
    self.f_pd_aao_dlt(_in_m_brownian)
    self.f_cfs_gao_dlt()
    _gao_dlt_mean = numpy.mean(self.m_pd_gao_dlt)
    _aao_dlt_mean = numpy.mean(self.m_pd_aao_dlt)
    _var_gao_dlt = numpy.var(self.m_pd_gao_dlt, ddof = 1)
    _c = numpy.cov(self.m_pd_aao_dlt, self.m_pd_gao_dlt)[0,1]
        / _var_gao_dlt if _var_gao_dlt > 0 else 0
    self.aao_kemna_vorst_dlt = _aao_dlt_mean
        - _c * (_gao_dlt_mean - self.cfs_gao_dlt)
```

A Appendix

A.1 Proof of 5.1.1

$$A^G(0) = e^{-RT} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right]$$

As $S(t) = S(0) \exp \left\{ \left(R - \frac{1}{2} \sigma^2 \right) t + \sigma \tilde{W}(t) \right\}$, then $\ln S(t) = \ln S(0) + \left(R - \frac{1}{2} \sigma^2 \right) t + \sigma \tilde{W}(t)$ and

$$\begin{aligned} \frac{1}{T} \int_0^T \ln S(t) dt &= \frac{1}{T} \left(T \ln S(0) + \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \sigma \tilde{X}(T) \right) \\ &= \ln S(0) + \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \end{aligned}$$

where $\tilde{X}$ is a random variable under $\tilde{\mathbb{P}}$ such that $\tilde{\mathbb{E}} \tilde{X}(T) = 0$ and $\text{Var}[\tilde{X}(T)] = \frac{T^3}{3}$.

$$\begin{aligned} \tilde{\mathbb{E}} \left[\left(\exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} - K \right)^+ \right] &= \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} - K \right)^+ \right] \\ &= \tilde{\mathbb{E}} \left[\left(S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} - K \right) \mathbb{I}_{\{S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} > K\}} \right] \\ &= \tilde{\mathbb{E}} \left[S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} \mathbb{I}_{\{S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T + \frac{\sigma}{T} \tilde{X}(T) \right\} > K\}} \right] \\ &\quad - K \cdot \tilde{\mathbb{P}} \left\{ \tilde{X}(T) > \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} = E_1 \end{aligned}$$

Here, take $\lambda_T = \frac{T}{\sigma} \ln \left(\frac{K}{S(0)} \right) - \frac{T^2}{2\sigma} \left(R - \frac{1}{2} \sigma^2 \right)$

$$\begin{aligned} E_1 &= \tilde{\mathbb{E}} \left[S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} \exp \left\{ \frac{\sigma}{T} \tilde{X}(T) \right\} \mathbb{I}_{\{\tilde{X}(T) > \lambda_T\}} \right] - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \\ &= S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T \right\} \frac{1}{\sqrt{\frac{2}{3} \pi T^3}} \int_{\lambda_T}^{\infty} e^{\frac{\sigma}{T} x} e^{-\frac{3x^2}{2T^3}} dx - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \\ &= S(0) e^{\frac{1}{2} RT - \frac{1}{12} \sigma^2 T} \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right) \end{aligned}$$

where, defining $\gamma_T = \exp \left\{ -\frac{1}{2} RT - \frac{1}{12} \sigma^2 T \right\}$, and substituting into original equation for $A^{(G)}(0)$ gives:

$$A^G(0) = S(0) \gamma_T \left(1 - N \left(\lambda_T, \frac{\sigma T^2}{3}, \sqrt{\frac{T^3}{3}} \right) \right) - K e^{-RT} \left(1 - N \left(\lambda_T, 0, \sqrt{\frac{T^3}{3}} \right) \right)$$

Which was to be shown. ■

A.2 Linear approximation of Brownian motion (5.2.1)

As seen in 6.1, $w(t_i)$, which is a discrete step of a simulated Brownian motion. On the range $(w(t_i), w(t_{i+1}))$, $\widetilde{W}(t)$ is a linear function:

$$\widetilde{W}_{(w(t_i), w(t_{i+1}))}(t) = \frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i}(t - t_i) + w(t_i)$$

Substituting into the formula for $\bar{S}(T)$:

$$\begin{aligned} \bar{S}(T) &= \frac{1}{T} \int_0^T S(0) \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 \right) + \sigma \widetilde{W}_{(w(t_i), w(t_{i+1}))}(t) \right\} dt \\ &= \frac{S(0)}{T} \sum_{i=0}^{i_T} \int_{t_i}^{t_{i+1}} \exp \left\{ t \left(R - \frac{1}{2} \sigma^2 + \frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i} \cdot \sigma \right) - \sigma \left(\frac{w(t_{i+1}) - w(t_i)}{t_{i+1} - t_i} \cdot t_i - w(t_i) \right) \right\} dt \end{aligned}$$

Which gives the equation in form as used in 6.1:

$$\bar{S}(T) = \frac{S(0)}{T} \sum_{i=0}^{i_T} \left(\frac{1}{\alpha_i} e^{\beta_i} (e^{\alpha_i t_{i+1}} - e^{\alpha_i t_i}) \right)$$

A.3 Proof of 5.2.2

$$\bar{S}(T) = \exp \left\{ \frac{1}{T} \int_0^T \ln S(t) dt \right\} = S(0) \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \frac{\sigma}{T} \tilde{X}(T) \right\}$$

Hence, taking $\frac{\partial \bar{S}(t)}{\partial S(0)}$, get the obvious result:

$$\frac{\partial \bar{S}(T)}{\partial S(0)} = \exp \left\{ \frac{1}{2} \left(R - \frac{1}{2} \sigma^2 \right) T^2 + \frac{\sigma}{T} \tilde{X}(T) \right\} = \frac{\bar{S}(T)}{S(0)}$$

Which was to be shown. ■